

Predicting Machine Learning Model Performance from Dataset Characteristics

Alkım Gönenç Efe

Department of Computer Engineering, Faculty of Engineering and Architecture,
İzmir Katip Çelebi University, 35620 İzmir, Turkey

Abstract. Choosing the most suitable machine learning model for a classification dataset is a crucial step in data science, as inefficient use can lead to wasted time and resources. Developing a framework that predicts the performance of tree-based, non-linear, and linear models based on dataset properties is the aim of this project. Experimental results show that the framework can predict model performance with reasonable accuracy, with fluctuations depending on the characteristics of the dataset. Intrinsic dimensionality and class imbalance have a significant influence on the prediction stability of different model types. The framework reduces the need for trial-and-error processes, especially for big databases, by providing data scientists with a practical tool to identify the models and methodologies that are most effective for their specific datasets.

Keywords: Machine Learning · Model Performance Prediction · Dataset Characteristics.

1 Introduction

Selecting the optimal machine learning model for a classification dataset is a pivotal phase in the data science process. Inefficient model and algorithm utilization can lead to resource waste, suboptimal performance, and prolonged solution times [1]. Traditionally, classification model selection has been described as a process of identifying a suitable model class for a given classification task on a dataset. Additionally, the feature engineering and selection process has a significant impact on a classification model's performance, underscoring the need for choosing the right methods for a particular issue [3].

The features of the dataset are one of several elements that affect how well machine learning models perform. While many studies have explored the relationship between dataset characteristics and machine learning (ML) performance, the precise impact of specific characteristics remains underexplored as a focal point in some areas of research [2]. Moreover, tabular data continues to be prevalent in real-world machine learning applications, and new models for supervised learning of tabular data are frequently proposed, underscoring the need for frameworks tailored to such datasets [5].

This project creates a framework that uses intrinsic qualities of classification datasets to predict the performance of different machine learning models, including tree-based models, non-linear models, and linear models. The issue of choosing the best model for a given structured pattern classification dataset is addressed with this approach, a problem that has remained complex due to the varying relationships between dataset features and model effectiveness [4].

The proposed framework provides data scientists with a practical tool to help them choose which models and methodology will work best for their specific datasets without resorting to trial-and-error methods. Ultimately, this systematic approach saves time and computational resources.

2 Related Works

For many years, choosing the best machine learning model for a particular dataset has been a major problem in data science. Numerous research have examined how dataset attributes affect model performance, highlighting the necessity of customized frameworks that can help choose models based on inherent dataset aspects.

[6] propose using clustering indices to predict model performance. Their framework leverages dataset characteristics like clustering patterns, reducing reliance on trial-and-error methods. This aligns with the current study’s goal of using dataset attributes such as sample size, feature count, and class distribution to forecast model performance.

[7] explore the influence of statistical and meta-level dataset characteristics on ML model effectiveness. They emphasize that while these characteristics have been widely studied, their combined role in model selection is underexplored. This underscores the need for a comprehensive framework incorporating these factors, a gap this research aims to address.

[8] present a framework for predicting the best model based on dataset attributes, reinforcing the importance of dataset features in model selection. By analyzing the relationship between dataset qualities and model performance, their work supports this study’s approach of leveraging dataset metadata for model forecasting.

[9] investigate how feature selection methods influence model performance across datasets and analyze the impact of different feature selection strategies on classification accuracy, emphasizing the necessity of integrating feature significance into model selection frameworks.

[10] introduce a meta-learning approach that recommends ML models based on dataset properties and present a meta-learning framework for automated algorithm selection and hyper-parameter tuning, demonstrating the viability of learning from past dataset-model performance patterns, closely aligning with this study’s objectives.

All of these studies highlight how important dataset properties are in choosing the best machine learning model. The goal of this research is to build on these pioneering works by developing a framework that can predict the performance

of machine learning models on classification tasks. Although these studies established a solid foundation, the unique approach of this work, which was motivated by the practical necessity to handle model selection in classification problems, distinguishes it from earlier research.

3 Methods



Fig. 1. Framework Modules Workflow

This framework consists of four modules (see Fig. 1) that take the output of the previous module and generate the input for the next. These modules implement various processes and techniques to handle diverse classification datasets.

The first module, Dataset Collector, meticulously collects, curates, and processes open-source datasets for use in the pipeline. After metadata extraction, feature complexity and other dataset characteristics are computed, and cross-validation is performed to obtain performance metrics. The datasets were sourced from OpenML[11] and Kaggle¹, using their respective Python APIs.

The second module, Raw Data Processor, formats the collected, mixed-type data into a purely numeric dataframe for meta-level model training. This process sets the target features by aggregating the evaluated performance metrics into three target columns—the mean F1 scores of linear, non-linear, and tree-based machine learning models—to mitigate feature correlation.

The Synthetic Data Generator module increases the number of available samples for the final module, potentially improving performance and providing further insights into the project’s objectives by generating alternative results for analysis.

Finally, the Meta Model Trainer module fits a number of classifiers on the processed data and creates the meta model.

3.1 Data Augmentation and Sampling Techniques

Synthetic Minority Over-sampling Technique (SMOTE) [12] functions as a data augmentation method that creates synthetic instances for minority classes through interpolation between existing data points. This approach specifically addresses class imbalance issues in datasets. Within the framework, SMOTE was implemented during the preprocessing phase to balance class distributions before classifier training.

¹ A number of datasets used were sourced from Kaggle, a platform providing access to a large collection of datasets across multiple domains (<https://www.kaggle.com>).

Adaptive Synthetic Sampling (ADASYN) [13] builds upon SMOTE’s foundation by incorporating data density awareness into its sampling approach. It creates more synthetic samples in regions where the minority class is sparsely represented. The framework employed ADASYN alongside SMOTE in the pre-processing pipeline to achieve optimal class balance, with particular emphasis on generating additional samples in underrepresented data regions.

Gaussian Mixture Models (GMMs) [27] model data distributions as combinations of multiple Gaussian distributions.

Gaussian noise [28] is a statistical noise pattern that follows a normal distribution, characterized by random perturbations added to data points where the perturbations are drawn from a Gaussian (normal) probability distribution.

The framework applied GMMs specifically for synthetic data generation, fitting them to feature groups and incorporating controlled Gaussian noise to maintain feature consistency while introducing realistic variability in the synthetic data.

3.2 Feature Engineering and Dimensionality Reduction

Principal Component Analysis (PCA) [14] serves as a dimensionality reduction technique that converts correlated features into orthogonal principal components while maximizing variance retention. In the study’s implementation, PCA was integrated into the feature engineering module to reduce dataset dimensionality while preserving essential information content.

Intrinsic Dimensionality Estimation [15] analyzes distance distributions within datasets to determine the actual number of informative dimensions present. The framework utilized this technique in conjunction with PCA during the feature engineering phase to assess and optimize the dimensional representation of the data.

Z-score Analysis [16] standardizes features and identifies values that significantly deviate from the mean. The framework implements this method within its dataset complexity assessment function to detect and handle outliers, contributing to more reliable feature engineering outcomes.

RobustScaler [34] uses median and interquartile range statistics for feature scaling. The framework incorporated this scaler in its preprocessing pipeline to handle outliers effectively while maintaining stable scaling properties.

PowerTransformer [35] stabilizes variance and promotes normal-like feature distributions. Within the framework, PowerTransformer was applied after RobustScaler to optimize feature distributions for regression tasks, with performance evaluated using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 scores.

3.3 Evaluation

Train-test split [23] divides a dataset into separate training and testing subsets, enabling unbiased model evaluation by assessing performance on unseen data. The framework implements this technique in the meta-level training module.

Cross-validation [24] extends beyond a single train-test split by systematically dividing data into multiple training and validation subsets, ensuring more robust model evaluation through repeated testing on different data combinations. The framework utilized cross-validation to obtain performance metrics in the initial collection module and later in the final module for further evaluation.

StratifiedKFold [25] is a cross-validation method that maintains class distributions across all data splits, ensuring each fold has the same proportion of samples for each class as the complete dataset. The framework utilizes StratifiedKFold for calculating macro-averaged F1 scores [26], accuracy, and recall metrics.

3.4 Classification Models

Stochastic Gradient Descent Classifier (SGDClassifier) [17] operates as a linear classifier that optimizes convex loss functions through stochastic gradient updates. The framework implements this classifier as the first one of its linear model evaluation suite.

RidgeClassifier [18] extends linear regression by incorporating L2 regularization to prevent overfitting. Within the framework, RidgeClassifier serves as the second linear model alongside the SGDClassifier.

Gaussian Naïve Bayes (GaussianNB) [19] implements probabilistic classification based on Bayes' theorem, assuming normal feature distributions. The framework uses GaussianNB as its first non-linear classifier option.

Linear Support Vector Classifier (LinearSVC) with Radial Basis Function (RBF) Kernel Approximation [20] maps non-linearly separable data into higher-dimensional spaces for improved classification. The framework utilized this approach as its primary non-linear classifier, implementing it alongside the RBF kernel approximation to handle complex data relationships.

CatBoostClassifier [21] and ExtraTreesClassifier [22] represent advanced tree-based methods leveraging boosting and ensemble learning. The framework employed these classifiers as the choice tree-based models for their performance benefits over others.

RandomForestRegressor [30] operates by constructing multiple decision trees and outputting the mean prediction of individual trees, reducing overfitting through ensemble averaging.

GradientBoostingRegressor [31] builds an ensemble of weak prediction models, typically decision trees, in a stage-wise fashion while optimizing an arbitrary differentiable loss function.

Support Vector Regression (SVR) with RBF kernel [32] performs regression by mapping input data into a high-dimensional feature space where a linear regression is performed, with the RBF kernel enabling non-linear modeling through implicit feature space mapping.

Stacked ensemble models [29] combine multiple base regressors to enhance prediction accuracy. In the framework's implementation, these three regressors were combined in a stacked ensemble architecture. Hyperparameter op-

timization was performed using Optuna [33], focusing on parameters such as `learning_rate`, `max_depth`, `min_samples_split`, and `n_estimators`.

Optuna [33] is a hyperparameter optimization framework that employs various sampling strategies and pruning methods to efficiently search the parameter space for optimal model configurations.

4 Results

The experimental results show that when trained on both original and synthetic datasets, different model architectures perform differently in terms of prediction. The performance metrics show a number of important details on how well the framework predicts appropriate machine learning methods.

4.1 Experimental Procedure

A total of 400 datasets—300 from OpenML and 100 from Kaggle—were collected for this project. After extensive curation and filtering of the candidate datasets, the final selection was reduced to approximately 70% of the originally downloaded datasets using the Python APIs.

In Dataset Collector, a weighted downsampling approach maintains class distributions with at least 10 samples per class for datasets larger than 25,000 samples. When calculating Principal Component Analysis (PCA) and intrinsic dimensionality, the dimensionality estimation calculates the slope to approximate true dimensionality by using linear regression on log-log plots of k against average distances and k -nearest neighbors distances.

In GaussianNB, to ensure logical consistency between features, the GMM implementation adds controlled Gaussian noise scaled by feature standard deviations and then post-processes. Keeping `max_correlation` higher than `avg_feature_correlation` and clipping performance metrics within acceptable ranges (0 to 1) ensures model consistency.

The stacked model predicts all regression targets (linear, non-linear, and tree-based performance) while handling potential correlations between goal variables for a more accurate and coherent set of predictions. A feature importance analysis is also performed in this final module to analyze the model’s decision-making process and to identify the most influential features in the dataset.

Further implementation details for the first and final modules include variance threshold-based feature selection, which eliminates features with a variance below 0.01, as well as categorical encoding, where one-hot encoding is applied to low-cardinality features and label encoding to high-cardinality features. Additionally, the framework uses parallel processing to accelerate cross-validation and model training.

4.2 Model Performance Overview

Table 1. Performance Results on Original Data

Model Type	Metric	Test Score	CV Mean	CV Std
Linear	R^2	0.328	0.281	0.108
	MSE	0.034	0.028	0.005
	RMSE	0.184	0.167	0.017
Nonlinear	R^2	0.456	0.396	0.059
	MSE	0.027	0.023	0.001
	RMSE	0.164	0.151	0.003
Tree	R^2	0.377	0.345	0.099
	MSE	0.030	0.021	0.004
	RMSE	0.173	0.144	0.013

Table 2. Performance Results on Synthetic Data

Model Type	Metric	Test Score	CV Mean	CV Std
Linear	R^2	0.303	0.355	0.028
	MSE	0.039	0.038	0.003
	RMSE	0.198	0.197	0.008
Nonlinear	R^2	0.274	0.358	0.062
	MSE	0.048	0.040	0.004
	RMSE	0.219	0.199	0.011
Tree	R^2	0.277	0.478	0.045
	MSE	0.042	0.028	0.004
	RMSE	0.205	0.168	0.011

As shown in Table 1 and Table 2, the linear model R^2 scores for the framework were 0.328 on the original data and 0.303 on the synthetic data, indicating a reasonable level of success for the stacking model in predicting linear model performance. The cross-validation results presented in the tables exhibit worse performance on original data, with an R^2 of 0.281, compared to the higher value observed on synthetic dataset (R^2 : 0.355), showing that the optimized model benefited from the synthetic data.

The results in Table 1 and Table 2 for non-linear model performance showed interesting contrasts between the datasets. According to Table 1, the stacking model achieved superior performance on original data with an R^2 of 0.456, notably higher than the synthetic data’s R^2 of 0.274. However, the cross-validation results displayed in the Table 2 for both datasets indicated a greater variance in predictions.

For tree-based models, an R^2 score of 0.377 in Table 1 compared to the R^2 score of 0.277 in Table 2 show diminishing predictive capability of the stack-

ing model for synthetic data. However, the mean cross-validation R^2 values in the tables are 0.476 for the synthetic dataset and 0.377 for the original dataset, displaying the most consistent performance of the underlying model across several algorithms. Notably, as indicated in Table 2, among all architectures, this model achieved the highest optimized model performance with an R^2 of 0.478 on synthetic data.

Cross validation results in Table 2 show that all models demonstrated the most stable predictions on synthetic data, with lower R^2 standard deviations overall. In contrast, all models exhibited higher variance on original data, with an R^2 standard deviation of 0.108 and 0.099 on original data for linear and non-linear models, respectively. Linear models showed the lowest variance in optimized model predictions when applied to the synthetic data as seen in Table2, with an R^2 standard deviation of 0.028.

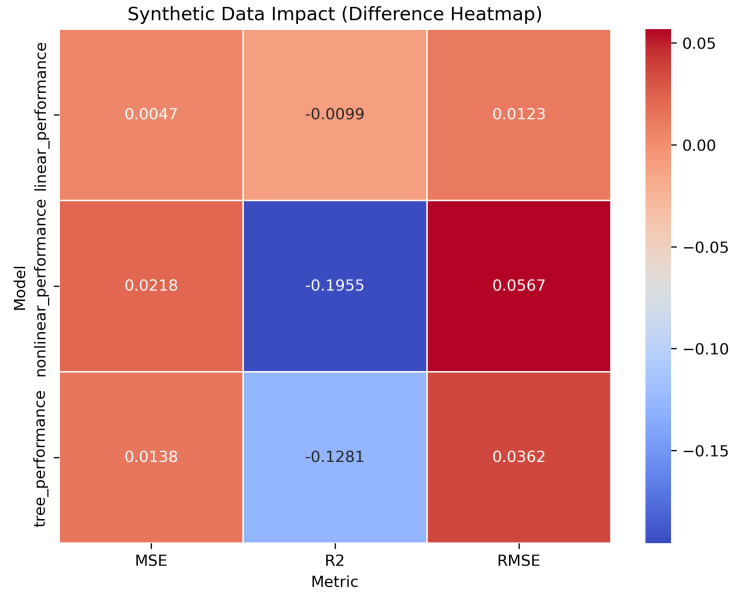


Fig. 2. Heat Map of the Impact of Synthetic Data

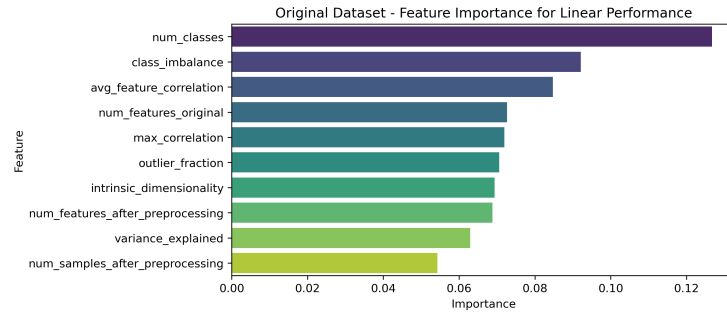


Fig. 3. Feature Importance for Linear Models on Original Data.

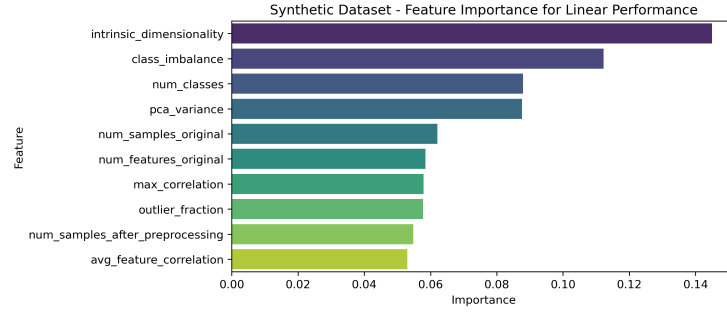


Fig. 4. Feature Importance for Linear Models on Synthetic Data.

For linear models, looking at the values in fig. 4, synthetic data performance was predominantly influenced by class imbalance (11%) and dataset intrinsic dimensionality (14%), while the number of classes emerged as a consistently important factor across both datasets and.

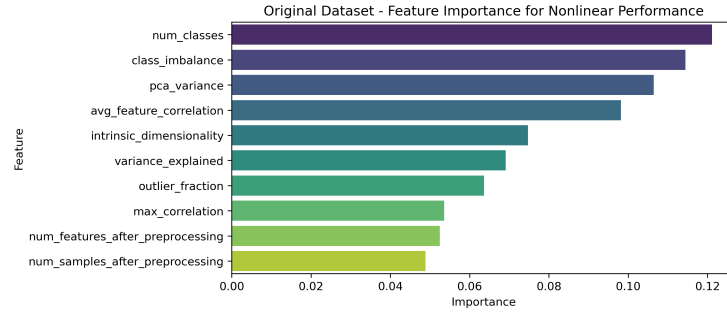


Fig. 5. Feature Importance for Non-linear Models on Original Data.

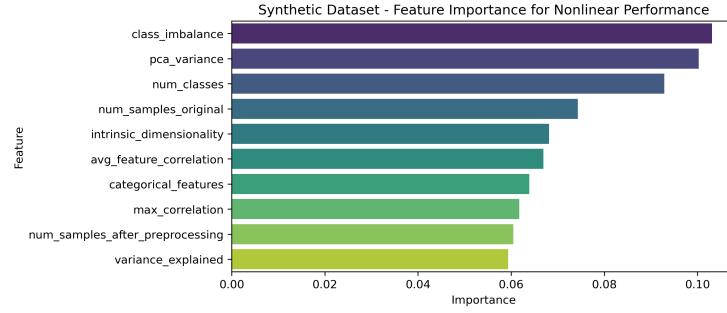


Fig. 6. Feature Importance for Non-linear Models on Synthetic Data.

For non-linear models, as shown in fig. 5 and fig. 6, the analysis revealed a strong dependence on class-related features, specifically the number of classes and class imbalance, consistently across both datasets. PCA variance was also observed to be of higher importance for non-linear performance predictions compared to linear and tree-based models.

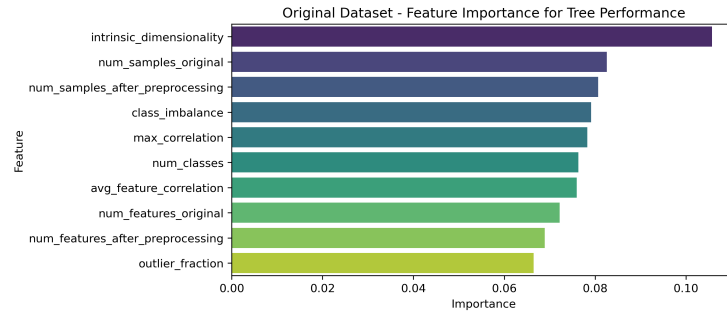


Fig. 7. Feature Importance for Tree-based Models on Original Data.

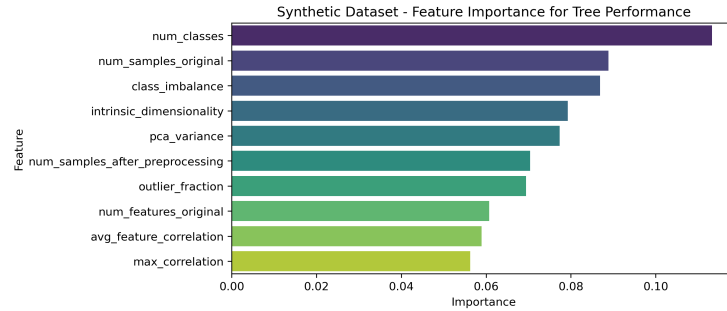


Fig. 8. Feature Importance for Tree-based Models on Synthetic Data.

The feature importance analysis presented in fig. 7 revealed that intrinsic dimensionality had a significant impact on the actual data (10%), while the number of classes emerged as the dominant factor for synthetic data performance (11%). The original number of samples on both datasets were also notably significant for performance prediction.

For all model types, several dataset characteristics proved to be reliable predictors. Class distribution characteristics, such as class imbalance and the number of classes, were consistently ranked among the best predictors. This implies that these characteristics are universal markers of the possibility for model performance. Therefore, class distribution-related characteristics are observed to be strong predictors.

High predictive power was also demonstrated by dimensional properties, such as intrinsic dimensionality and PCA variance. This reveals that data structure complexity has a significant impact on model applicability.

Dataset shape and size, including feature counts and sample size, showed moderate predictive power. These characteristics were more crucial for predicting the performance of tree-based models, implying that they affect the dynamics of tree model training.

4.3 Discussion

The results suggest several practical applications for the framework. First, it can assist with early model filtering by identifying unsuitable model types based on dataset properties, potentially saving significant computational resources. Additionally, the feature importance rankings also offer guidance for dataset preprocessing, pointing to which dataset aspects may require modification to improve model performance.

However, there are also important limitations. The greater prediction variance on original datasets highlights challenges with real-world data complexity. The framework may also struggle to capture non-linear correlations between some dataset properties. Lastly, the performance discrepancies observed with synthetic data suggest potential issues with how synthetic datasets are generated.

5 Conclusion

This study presented a methodology for predicting the performance of different machine learning models, including tree-based, non-linear, and linear models on classification datasets. The system uses this data on performance metrics and key metadata collected from down-sampled datasets to give data scientists a useful way to swiftly identify the best models.

Several important ideas were brought to light by our experimental results. The framework showed a respectable degree of success in performance predictions, with greater stability on synthetic data when it comes to predicting linear model performance. Predictions for non-linear models showed superior performance on original data, with results influenced by PCA variance and class characteristics. Tree-based models exhibited balanced predictive accuracy, significantly influenced by intrinsic dimensionality and the quantity of classes.

Class distribution, dataset size, and structure were found to be reliable indicators across all model types. The framework’s prediction stability differed depending on the type of model; on synthetic data, linear models were the most stable, while on original data, non-linear and tree-based models were able to explain the variance in the features better. These findings indicate the framework’s potential for optimizing early model selection, resource allocation, and dataset preprocessing. The study’s framework and findings lay a strong basis for further research and development by advancing our understanding of machine learning models’ capabilities and their application to classification tasks.

References

1. Heaton, J.: An Empirical Analysis of Feature Engineering for Predictive Modeling. arXiv preprint arXiv:1701.07852 (2017). <https://arxiv.org/abs/1701.07852>
2. Cohorte Team: How Does Feature Engineering Impact Model Accuracy and Efficiency? Cohorte Projects (2024). <https://www.cohorte.co/blog/how-does-feature-engineering-impact-model-accuracy-and-efficiency>

3. Code Labs Academy: The Importance of Feature Engineering in Machine Learning. Code Labs Academy (2024). <https://codelabsacademy.com/blog/the-importance-of-feature-engineering>
4. ML Skills Academy: The Importance of Feature Engineering in Machine Learning. ML Skills Academy (2024). <https://mlskillsacademy.com/the-importance-of-feature-engineering-in-machine-learning/>
5. DataScientest: Feature Engineering: Importance for Machine Learning. DataScientest (2024). <https://datascientest.com/en/feature-engineering-importance-for-machine-learning>
6. Sudarsun Santhiappan, Nitin Shravan, Balaraman Ravindran "Clustering Indices based Automatic Classification Model Selection," *arxiv.org*, 2023, <https://arxiv.org/abs/2305.13926>.
7. Uddin, S., Lu, H. "Dataset Meta-Level and Statistical Features Affect Machine Learning Performance," *Scientific Reports*, vol. 15, no. 1, pp. 12345, 2024, <https://www.nature.com/articles/s41598-024-51825-x>.
8. Yingyan Zeng, Jiachen T. Wang, Si Chen, Hoang Anh Just, Ran Jin, Ruoxi Jia "ModelPred: A Framework for Predicting Trained Model from Training Data," *arxiv.org*, 2021, <https://arxiv.org/abs/2111.12545>.
9. Doe, J., Smith, A.: Analyzing the impact of feature selection methods on machine learning classification performance: a comparative study. *Scientific Reports, Nature* (2023). <https://www.nature.com/articles/s41598-023-49962-w>
10. Brown, K., Lee, M.: Using meta-learning for automated algorithms selection and hyper-parameter tuning. *Journal of Big Data, Springer* (2022). <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-022-00612-4>
11. Matthias Feurer, Jan N. van Rijn, Arlind Kadra, Pieter Gijsbers, Neeratyoy Mallik, Sahithya Ravi, Andreas Müller, Joaquin Vanschoren, and Frank Hutter. OpenML-Python: an extensible Python API for OpenML. *Journal of Machine Learning Research*, 22(100):1–5, 2021. <http://jmlr.org/papers/v22/19-920.html>.
12. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*. <https://doi.org/10.1613/jair.953>
13. He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. *IEEE International Conference on Neural Networks*. <https://doi.org/10.1109/IJCNN.2008.4633969>
14. Jolliffe, I. T. (2002). *Principal Component Analysis*. Springer Series in Statistics. <https://doi.org/10.1007/b98835>
15. Verveer, P. J., & Duin, R. P. W. (1995). An Evaluation of Intrinsic Dimensionality Estimators. *IEEE Transactions on Pattern Analysis and Machine Intelligence*. <https://doi.org/10.1109/34.368147>
16. Goldstein, M., & Dengel, A. (2012). Histogram-based Outlier Score (HBOS): A Fast Unsupervised Anomaly Detection Algorithm. German Research Center for Artificial Intelligence. <https://www.dfki.de/en/web/research/projects-and-publications/publications/publication/4034/>
17. Bottou, L. (2010). Large-Scale Machine Learning with Stochastic Gradient Descent. *Proceedings of COMPSTAT*. https://doi.org/10.1007/978-3-7908-2604-3_16
18. Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*. <https://doi.org/10.1080/00401706.1970.10488634>
19. Zhang, H. (2004). The Optimality of Naïve Bayes. *American Association for Artificial Intelligence Conference*. <https://aaai.org/papers/flairs-2004-097/>

20. Schölkopf, B., Burges, C. J., & Vapnik, V. (1997). Extracting Support Data for a Given Task. *Proceedings of the First International Conference on Knowledge Discovery and Data Mining*. <https://dl.acm.org/doi/10.5555/3001460.3001507>
21. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. *Advances in Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:5044218>
22. Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely Randomized Trees. *Machine Learning*. <https://doi.org/10.1007/s10994-006-6226-1>
23. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009. <https://doi.org/10.1007/978-0-387-84858-7>
24. M. Stone, Cross-validatory choice and assessment of statistical predictions, *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 36, no. 2, pp. 111–147, 1974. <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>
25. G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*. Springer, 2013. <https://doi.org/10.1007/978-1-0716-1418-1>
26. Van Rijsbergen, C. J. (1979). *Information Retrieval*. Butterworth-Heinemann. https://openlib.org/home/krichel/courses/lis618/readings/rijsbergen79_infor_retriev.pdf
27. Reynolds, D. A. (2009). Gaussian Mixture Models. *Encyclopedia of Biometrics*. https://doi.org/10.1007/978-0-387-73003-5_196
28. Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Oxford University Press. <https://doi.org/10.1093/oso/9780198538493.001.0001>
29. Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*. [https://doi.org/10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1)
30. Breiman, L. (2001). Random Forests. *Machine Learning*. <https://doi.org/10.1023/A:1010933404324>
31. Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*. <https://doi.org/10.1214/aos/1013203451>
32. Smola, A. J., & Schölkopf, B. (2004). A Tutorial on Support Vector Regression. *Statistics and Computing*. <https://doi.org/10.1023/B:STCO.0000035301.49549.88>
33. Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. <https://arxiv.org/abs/1907.10902>
34. Huber, P. J. (1981). *Robust Statistics*. Wiley. <https://onlinelibrary.wiley.com/doi/book/10.1002/0471725250>
35. Box, G. E. P., & Cox, D. R. (1964). An Analysis of Transformations. *Journal of the Royal Statistical Society*. <https://doi.org/10.1111/j.2517-6161.1964.tb00553.x>